

Technical Cheat Sheet

Every term, model, method, dataset, and training decision in the project — defined and explained

1. Glossary of terms

Grouped by theme. Each entry: what it is, and how it is used in this project specifically.

1.1 Representations and spaces

Term	Definition and role in this project
Embedding / representation	A vector of numbers a model produces for an input, positioning it in a geometric space where distance ~ semantic dissimilarity. Everything in this project compares such vectors: 768-d hidden states (LLMs), 512-d MobileCLIP image vectors, 768-d SigLIP image/text vectors.
Activation / hidden state	The output of a specific layer for a specific input. LLMs produce one per token per layer; Experiment A stores one vector per passage per layer via mean pooling.
Mean pooling	Averaging token-level vectors over the sequence (masked to real tokens) to get one fixed-size vector per text. Lossy — discards word order — which makes Experiment A's similarity estimates conservative.
L2 normalization	Scaling a vector to unit length. After it, dot product = cosine similarity. All retrieval vectors in Experiment B are L2-normalized; the Core ML adapter normalizes its output.
Cosine similarity	Angle-based similarity in $[-1, 1]$, blind to vector length. The metric Qdrant search uses, hence the metric the adapter is judged by.
Anisotropy	Unequal spread of a space's variance across directions. MobileCLIP's and SigLIP's spaces differ in anisotropy — the reason rotation-only alignment (Procrustes) failed while full linear (ridge) succeeded.
Isomorphic (up to linear map)	Two spaces containing the same information in different coordinates, convertible by one matrix. The hypothesis asserts trained models' spaces are approximately this.
Permutation symmetry	Reordering a layer's neurons (with consistent rewiring) changes weights completely but not the function. The reason raw weights of independent models cannot be compared, averaged, or transplanted.

1.2 Measurement and statistics

Term	Definition and role in this project
Gram matrix	For n inputs, the $n \times n$ matrix of pairwise similarities as one model sees them — a coordinate-free fingerprint of the representation. CKA compares two models' Gram matrices.
CKA (Centered Kernel Alignment)	Similarity in $[0,1]$ between two representation sets, invariant to rotation, permutation, and uniform scaling — exactly the symmetries that leave a network's function unchanged. Used in A2 for every layer pair.
R^2 (coefficient of determination)	Fraction of the target's variance a prediction explains; 1 = perfect, 0 = as good as predicting the mean, negative = worse than the mean (seen in the starved run's random baseline). Used in A3 and B2 on held-out data.
Recall@K	Fraction of queries whose correct item appears in the top K results. The operational gate of Experiment B: adapter must keep $\geq 90\%$ of the ceiling's Recall.
Held-out / train-eval split	Fitting on one subset (75%) and measuring on untouched data (25%), so scores reflect generalization, not memorization. The same fixed split is reused across B2/B3/MLP for direct comparability.

Term	Definition and role in this project
Baseline (floor) / ceiling	The score with no learning (random weights; raw cross-space retrieval) vs the best achievable (native SigLIP). Every result in the project is read as its position between the two.
Sample-to-dimension ratio	Training rows per input dimension of a fitted map. At 1.6 (run 1) ridge is starved and systematically understates alignment; at ~10 (run 2) estimates stabilize. The project's key methodological lesson.
Overfitting / early stopping	Fitting noise beyond signal; prevented in the MLP comparison by keeping the checkpoint with best held-out cosine (reached at epoch 30 of 300).

1.3 Maps and training

Term	Definition and role in this project
Reparameterization (structural)	Folding a multi-branch training-time block (parallel convs + BatchNorm + identity) into a single equivalent convolution at inference. Used throughout MobileCLIP-S1; the reason its deployed graph is simpler and faster than its printed module tree, and why the ANE compiles it efficiently.
Identity normalization	A preprocessing Normalize step with mean 0 and std 1, i.e. no change. MobileCLIP-S1 uses it (raw [0,1] pixels); replicating the pipeline incorrectly on device silently invalidates the adapter's calibration.
Adapter (linear map)	Here: a single 512×768 matrix W translating MobileCLIP image vectors into SigLIP space: $y = L2norm(W \cdot x)$. 1.6 MB fp32 / 0.79 MB fp16. The project's deliverable.
Ridge regression	Linear least squares with an L2 penalty $\alpha \cdot I$ stabilizing the inverse: $W = (X^T X + \alpha I)^{-1} X^T Y$. Closed-form — solved exactly in one line, no gradient descent. Fits both the stitching maps (A3) and the adapter (B2) in seconds.
Closed-form solution	An answer computed by formula rather than iterative optimization. Why 'training' the adapter costs seconds and is exactly reproducible.
Orthogonal Procrustes	The best pure rotation aligning two point sets, from the SVD of $X^T Y$. Tests the stronger 'rigidly same shape' claim; succeeded implicitly in A (spaces near-rotational), failed in B ($R^2 = 0.038$) due to anisotropy differences.
MLP (multi-layer perceptron)	A small neural network (here 512->1024->768 with GELU) used as a controlled upgrade attempt. Its +0.003 non-improvement proved the linear map optimal.
GELU	A smooth activation function standard in transformers; the nonlinearity inside the comparison MLP.
Cosine loss	$1 - \cos(\text{prediction}, \text{target})$; trains the MLP directly on the quantity retrieval cares about, rather than MSE.
Model stitching	Connecting layer i of model A to layer j of model B through a small trained map and measuring the penalty. A3 is stitching with closed-form ridge and R^2 as the penalty measure.
Fine-tuning vs from-scratch	Fine-tuning updates an existing checkpoint (coordinates inherited); from-scratch starts from random init (coordinates independent). Both Experiment A models are from-scratch — the property that makes their alignment evidence of convergence.

1.4 Systems and deployment

Term	Definition and role in this project
Qdrant / vector database	A database that stores embeddings and answers nearest-neighbor queries. The target architecture: one collection, 768-d, cosine metric, written by both tiers.
HNSW	Hierarchical Navigable Small World — the graph index structure making approximate nearest-neighbor search fast; one shared graph instead of two is part of the efficiency payoff.

Term	Definition and role in this project
Core ML / .mlpackage	Apple's on-device inference format and runtime. B4 exports the adapter as an .mlpackage (MatMul + L2-normalize) via coremltools' PyTorch->MIL conversion.
Neural Engine (ANE)	The iPhone's NPU; runs fp16 Core ML models at very low energy. Executes both MobileCLIP and the adapter.
fp16 / fp32	16- vs 32-bit floating point. The adapter ships fp16 (0.79 MB, precision loss < 1e-3 — negligible for cosine ranking).
DATA_DIR convention	All pipeline stages read/write artifacts (activations.npz, pairs.npz, adapter.npz) under one configurable directory — on Colab, a mounted Google Drive path — making every stage stateless and VM-restart-proof.
Streaming / parquet datasets	Hub-hosted columnar data readable without custom loader scripts. The project migrated to parquet (wikitext) and direct official downloads (COCO) after HF deprecated script-based datasets mid-project.

2. Models: full profiles

Model	Architecture & size	Training data / objective	Role here
GPT-2 (124M)	Decoder-only transformer, 12 blocks, d=768, learned positional embeddings, GPT-2 BPE tokenizer (50,257 vocab). OpenAI, 2019.	WebText (~40 GB of Reddit-linked pages); next-token prediction.	Experiment A, model A. Frozen; only hidden states read.
Pythia-160M	GPT-NeoX decoder-only transformer, 12 blocks, d=768, rotary positional embeddings, NeoX tokenizer. EleutherAI, 2023.	The Pile (~825 GB, 22 diverse sources); next-token prediction. Trained from scratch — no shared ancestry with GPT-2.	Experiment A, model B. Frozen. The LM-head weight (embed_out) is discarded on load — the benign UNEXPECTED warning.
Random-weights Pythia	Identical architecture to Pythia-160M, weights freshly sampled from the init distribution; zero gradient steps.	None — that is the point.	The control: measures similarity obtainable from architecture + shared inputs alone. Trained-minus-random = effect of learning.
MobileCLIP-S1	Apple's efficient image-text model; hybrid conv-transformer image tower (FastViT/MC1), 21.5 M vision parameters verified by introspection; channel pyramid 64-128-256-512 with 4/12/20/4 blocks per stage; attention only in the final stage; embedding d=512. Designed for the ANE; loaded via open_clip (pretrained 'datacompdr').	Contrastive image-text training on DataCompDR (reinforced web image-text pairs).	The phone-tier encoder. Frozen; source space of the adapter.
SigLIP 2 (base, patch16-224)	Google's vision-language model; ViT-Base image tower (16px patches, 224px input), paired text tower; both embeddings d=768; ~1.5 GB checkpoint.	Sigmoid-loss contrastive image-text training (per-pair logistic objective instead of softmax) — the reason its absolute cosines run low (~0.15 diagonal) while ranking stays sharp.	The server-tier encoder and the target space. Frozen; its text tower produces all retrieval queries and is unaware the adapter exists.

Table: the five models. 'Frozen' = no weights updated anywhere in the project.

Encoders, precisely: an *encoder* is a model (or tower) mapping raw input to an embedding. The project uses four: GPT-2 and Pythia as text encoders (reading all 13 hidden-state levels, not just the last), MobileCLIP's image tower, and SigLIP's image + text towers. Decoding/generation is never used anywhere.

2.1 Inside MobileCLIP-S1 (verified by introspection)

Two structural facts matter for this project. **(a) Reparameterizable convolutions.** Nearly every block is a MobileOne/RepMixer unit holding parallel branches during training (3x3 conv + 1x1 conv + identity, each with BatchNorm) that fold algebraically into a single convolution at inference - training-time capacity at deployment-time cost. BatchNorm rather than LayerNorm throughout is what makes the fold possible. Attention appears only in the last stage (4 blocks, QKV 512 -> 1536) where the grid is already 8x8, so global mixing is nearly free. **(b) A one-layer bottleneck.** The trunk carries 1024 channels until a single final projection Linear(1024 -> 512) emits the embedding - so the capacity gap that limits R@1 is concentrated in one layer, and a wider-output sibling (MobileCLIP-B, MobileCLIP2) is the predicted remedy.

2.2 Preprocessing - a silent-failure trap

MobileCLIP-S1's transform is: bilinear Resize(256), CenterCrop(256x256), tensor conversion, and **Normalize(mean=(0,0,0), std=(1,1,1)) - an identity operation**. The model consumes raw [0,1] pixels, unlike standard CLIP models that subtract dataset means. Since the adapter matrix W is fitted against the embedding distribution this exact pipeline produces, replicating it inexactly on iOS (for example by copying conventional CLIP normalization constants) shifts every vector and degrades retrieval with no error raised. The iOS client must reproduce `resize-256 / center-crop / [0,1]` scaling and nothing else.

3. Data inputs: what, why, how

Dataset	Contents & size used	Why chosen	How used
WikiText-103 (Salesforce/wikitext, train split)	Long-form Wikipedia articles; 10,000 passages > 100 chars (run 2; run 1 used 1,646 validation passages).	Diverse, clean English prose; parquet-hosted (survives the datasets-scripts deprecation). Content is out-of-domain for neither model, favoring neither.	Identical passages fed to GPT-2, Pythia, and the random control; truncated at 64 tokens; batch 32; mean-pooled per layer -> activations.npz (3 × [13, 10000, 768]).
COCO val2017 (official cocodataset.org zips)	5,000 photos with 5 human captions each; 4,000 image-caption pairs used (first caption per image); ~1 GB images + 240 MB annotations.	Natural photos + human English captions — the adapter's actual production domain; standard retrieval benchmark, so results are sanity-checkable against published numbers; plain zips immune to loader-script deprecations.	Each image encoded by both image towers; each caption by SigLIP's text tower -> pairs.npz (mob_img 4000×512, sig_img 4000×768, sig_txt 4000×768), split 3000 train / 1000 eval.

Input processing details: LLM inputs are tokenized by each model's own tokenizer (deliberately different — part of the independence claim); attention-masked mean pooling excludes padding. Images are preprocessed by each encoder's own pipeline (resize/normalize per its training recipe); captions are tokenized by SigLIP's tokenizer, padded to 64. All embeddings are L2-normalized at extraction so every later stage works in cosine geometry.

4. Methods, step by step

4.1 CKA computation (A2)

For layer i of model A (matrix X , 10000×768) and layer j of model B (Y): center columns; $CKA = ||Y^T X||_F^2 / (||X^T X||_F \cdot ||Y^T Y||_F)$ — algebraically the cosine between the two centered Gram matrices. Computed for all 13×13 pairs, twice (trained-vs-trained, trained-vs-random). Read as: high diagonal = same content developed in the same order through depth.

4.2 Linear stitching (A3)

Per matched layer pair: 75/25 split; standardize inputs; solve ridge $W = (X^T X + aI)^{-1} X^T Y$ with $a=1.0$ on train; report variance-weighted R^2 on eval. Layer matching is one-to-one here (both models have 13×768 states); the code supports proportional matching for unequal depths. The run-1 -> run-2 comparison changed only N (1,646 -> 10,000) and the source split.

4.3 Adapter fitting (B2)

Ridge on raw (unstandardized, already L2-normalized) pairs with $a=0.01$: $W = (X^T X + aI)^{-1} X^T Y$, $X = \text{mob_img}[\text{train}]$ (3000×512), $Y = \text{sig_img}[\text{train}]$. Procrustes variant: zero-pad X to 768, $\text{SVD}(X^T Y) = U S V^T$, $R = U V^T$. Both evaluated on the same 1,000 held-out images by R^2 and mean cosine-to-target.

4.4 Retrieval evaluation (B3)

Queries: SigLIP text embeddings of the 1,000 eval captions (exactly the production query path). Galleries: (A) native sig_img — ceiling; (B) $L2\text{norm}(\text{mob_img} @ W)$ — the system; (C) raw mob_img zero-padded — chance floor. Similarity = dot product (all unit-norm); ground truth is the diagonal (caption k matched to image k); $\text{Recall}@\{1,5,10\}$ = fraction of captions ranking their own image in top K . Gate: $B \geq 90\%$ of A per K .

4.5 MLP comparison (B2-bis)

`nn.Sequential(Linear 512->1024, GELU, Linear 1024->768)`; AdamW lr $1e-3$, weight decay $1e-4$; cosine LR schedule; batch 256; 300 epochs; loss = $1 - \text{cosine}(\text{normalized prediction}, \text{target})$; early stopping keeps the best held-out-cosine checkpoint (epoch ~30). Same split as B2. Result: +0.003 cosine, retrieval unchanged -> linearity established.

4.6 Core ML export (B4)

Wrap W in a bias-free torch Linear plus `F.normalize`; `torch.jit.trace` on a dummy (1×512); `coremltools.convert` with FLOAT16 precision, iOS16 target; save Adapter.mlpackage (0.79 MB). Conversion path: PyTorch graph -> MIL ops -> mlprogram. Parity check on macOS: $\max |\text{CoreML} - \text{numpy}|$ expected $< 1e-3$ (fp16).

5. What was trained — and what deliberately was not

Component	Trained?	How / why not
GPT-2, Pythia, MobileCLIP, SigLIP 2	No — frozen throughout	The entire point: interoperability without touching the models.
Random-weights Pythia	No — never trained by design	Control condition isolating the effect of learning.
13 stitching matrices (A3)	Closed-form fit (seconds)	Ridge regression; 'trained' only in the loosest sense — no epochs, no gradients.
Adapter W (B2)	Closed-form fit (seconds)	Ridge on 3,000 pairs; refittable in seconds whenever either encoder is upgraded.
Procrustes R (B2)	Closed-form fit (seconds)	SVD; tests the rotation-only hypothesis.

Component	Trained?	How / why not
Comparison MLP (B2-bis)	Yes — 300 epochs, ~2 min GPU	The only gradient training in the project, run deliberately as a control to prove it was unnecessary.

Total compute for the full study — extraction, two A-runs, all fits, MLP, export — is under one GPU-hour on a free Colab T4.

Addendum — Experimental design vocabulary

Terms used when discussing why Experiment A was run the way it was (see report Appendix C).

Term	Meaning
Train / validation / test split	Partitions of a dataset defined by how much information reaches the model: train (weights fitted on it), validation (guides the experimenter's choices), test (used once, for the honest final number). The roles presuppose a model being trained - they carry no meaning for frozen models used only to read activations.
Probe stimuli	Inputs fed to a frozen model purely to elicit internal activations for measurement, analogous to a stimulus set in a perception experiment. WikiText plays this role in Experiment A: it trains nothing.
Held-out evaluation	Measuring a fitted map on rows it never saw. A 768x768 matrix with 590K free parameters can memorize a small sample exactly; in-sample R2 would then be meaningless. Every R2 in this project is held-out.
Samples per dimension	Training rows divided by input dimensionality - the practical measure of statistical power for a linear fit. Below roughly 5, ridge shrinkage dominates and estimates become systematically pessimistic. Run 1: 1.6. Run 2: 9.8.
Shrinkage bias (starved regression)	When rows are scarce relative to parameters, the ridge penalty forces coefficients toward zero to stay stable. The fit generalizes poorly in one direction: it understates the true relationship. Negative held-out R2 on a control is its signature - the fit generalizes worse than predicting the mean.
Pre-registration	Fixing success criteria before seeing results, so that a verdict cannot be chosen after the fact. Used for both experiments; Appendix C also records the one sub-threshold that was missed.
Diagnosed confound	A specific, checkable reason a measurement may be wrong, identified before rerunning and paired with a prediction that distinguishes it from the alternative. What separates a correction from fitting the result.

Rule of thumb. Before accepting any negative result from a fitted map, check that the measurement had the power to show a positive one: rows per parameter first, then alignment of the two data matrices, then the pooling or preprocessing that produced them.

Cheat Sheet Addendum - E and F series

Terms, models and settings introduced after the original cheat sheet was built. Everything in Sections 1-4 of that document still applies.

A1. Vision-transformer pooling: CLS, patches, and cls+patch

A vision transformer does not read an image as a whole. It cuts it into a grid of square **patches** - DINOv2 uses 14x14 pixels, so a 224x224 image becomes 16x16 = 256 patches - and treats each as a token, like a word in a sentence. One extra learned token, **CLS**, is prepended; it carries no image content of its own and exists to attend over all the patches and compress them into a single summary. After the transformer, the hidden state is [257, d]: row 0 is CLS, rows 1..256 are the patches, each describing its own region as informed by the whole image.

POOLING	What is taken	Width	Why
cls	row 0 only - the model's own global summary	d	the token the training objective operated on; one vector, cheapest
cls+patch	row 0 concatenated with the mean of rows 1..256	2d	DINOv2's own linear-probe protocol. The patch mean keeps texture, spatial statistics and fine detail that the CLS compression discarded; neither subsumes the other

Consequences in the notebooks. The width doubles, so N_PAIRS auto-scales to hold rows-per-input-dimension near 10 (dinov2-large with cls+patch: width 2048, 23,755 pairs, 11.1 rows/dim). And because pooling changes *what information* the vector carries - not merely how many dimensions hold it - it is a genuine confound for the scale sweep: every encoder in a sweep must use the same pooling, or a gain cannot be attributed to capacity. Measured example: dinov2-base with cls reached 40.9 percent of ceiling and dinov2-large with cls+patch 58.3, but four factors differed between those runs, so the difference is not a capacity measurement.

A2. Regularization vs learning rate

Term	Where	What it does
alpha (ridge)	A3, B2, E-series: $W = (X^T X + \alpha I)^{-1} X^T Y$	Regularization strength - shrinks W toward zero before inversion. NOT a learning rate: there is no loop, no gradient and no schedule; the solve is closed-form
ALPHA_GRID	E-series	A search , not a schedule: each candidate is scored on a validation slice carved from TRAIN only, the winner is fixed, and one final fit is run on all training rows at that value
learning rate	F2 (1e-3, cosine-annealed), F3 (1e-4)	The genuine article - F trains artifacts by gradient descent. F3 uses a lower rate because it perturbs pretrained weights

Diagnostic value. How much alpha matters is itself a reading of the space: in the dinov2-large run every candidate scored 0.596 (22,755 rows against 2,048 dimensions is well-conditioned, so shrinkage is nearly irrelevant), while in E1.1 alpha = 10 produced R-squared -0.741 because GPT-2's near-degenerate space is severely ill-conditioned. Same grid, opposite sensitivity.

A3. Models added after the original sheet

Model	Architecture / size	Training	Role here
DINOv2 (small 22M / base 87M / large 304M)	ViT, patch 14; widths 384 / 768 / 1024 (doubled by cls+patch)	Self-supervised: self-distillation between two augmented crops plus masked-image modelling. No captions, no text tower, no cross-modal loss - text never enters the objective. Its dataset (LVD-142M) was curated using label signals, which biases which images were selected but never reaches the loss	The image arm of the E series, and the scale-sweep variable. Chosen precisely because a CLIP-family encoder would make the cross-modal test circular
bge-m3 (~568M)	XLM-RoBERTa backbone, 1024-d dense output, one size only	Contrastive retrieval training on ~1.2B multilingual text pairs; dense, sparse and multi-vector modes (only dense is used here). Never saw a pixel	The text arm of E1/E2. Its space is <i>shaped for cosine</i> , which E1.1 shows is not incidental
GPT-2 as a feature extractor	124M, 768-d, masked-mean pooled over ≤ 64 tokens	Next-token prediction only - never optimized for similarity. Used for inference, never generation	The E1.1 contrast arm: a text space with no similarity training. Effective rank 6.5 of 768, mean pair-cosine +0.999

A4. Terms introduced by the E/F results

Term	Meaning and measured anchor
matched ceiling	A jointly trained model scored on the same rows, same direction and same caption treatment as the system. Mismatching any of the three inflates the percentage: an averaged-caption system against a single-caption ceiling read 55.2 percent where the matched figure is 40.9
caption averaging	COCO supplies five human captions per image; averaging their embeddings cancels annotator idiosyncrasy. R-squared 0.278 $\rightarrow$ 0.511, R@1 0.160 $\rightarrow$ 0.358. The ceiling must be averaged too (SigLIP: 0.648 single, 0.875 averaged)
metric degeneracy	A similarity metric can be uninformative in a given geometry. In GPT-2's space (pair-cosine +0.999) cosine reads 1.000 for a map whose retrieval runs at 9.5 percent - the mirror of Experiment B, where R-squared understated a system whose cosine was honest
stitching penalty	$1 - R@K(\text{stitched}) / R@K(\text{ceiling})$: what a downstream consumer loses when fed a translated vector. Measured 0.049 at R@5 - near-transparent
relational loss	Matches the student's in-batch similarity (Gram) matrix to the teacher's, so training targets relational structure rather than coordinates. Used in F2/F3 alongside cosine
teacher-tagged checkpoint	A cache that records every setting affecting its contents (teacher id, encoder, pooling, width) and validates them on resume. Introduced after three separate silent failures from caches keyed on only part of the configuration
MATCH_DIM	Optional PCA projection of every encoder to a common width, so a scale sweep compares representation quality rather than fit capacity. Measured as a weak effect once N scales with width, since rows-per-parameter then stays constant

A7. What a transformer layer actually outputs (and what pooling does)

Loose phrasing to avoid: a layer does **not** produce 'a vector'. It produces **one vector per token**. A five-token passage through GPT-2 yields a grid of 13 layers x 5 tokens, each cell a 768-number vector - 65 vectors from one passage. Attention mixes the tokens within each layer (so the vector at position 'sang' carries context from 'the bird' before it), but nothing ever merges them into one; a transformer stage reads a set of token vectors and writes a set of the same shape.

Stage	Shape	What it is
layer output	[T, 768] per layer	one vector per TOKEN; T varies per passage
masked mean pooling	[768] per layer	average across the T real tokens, pads excluded from both the sum and the divisor. Done separately within each layer, so the layer axis survives - that axis is what Experiment A measures
one passage	13 x [768]	one vector per layer

Stage	Shape	What it is
the saved array	[13, 10000, 768]	13 layers x 10,000 passages x 768 coordinates. Row i is the SAME passage in GPT-2 and in Pythia

Worked masked mean. Take $T = 3$ real tokens plus one pad, and 4 coordinates instead of 768:

token	vector	mask
The	[2, 0, 4, 1]	1
bird	[0, 6, 2, 3]	1
sang	[4, 3, 0, 2]	1
<pad>	[9, 9, 9, 9]	0

Sum over real tokens only: $[2+0+4, 0+6+3, 4+2+0, 1+3+2] = [6, 9, 6, 6]$. Divide by the number of real tokens, 3, not by 4: **[2, 3, 2, 2]**. The pad row contributes nothing to the numerator and is excluded from the denominator - had it been included the result would be $[3.75, 4.5, 3.75, 3.75]$, contaminated by a token carrying no content.

Why pool at all. Two tokenizers do not agree on what the tokens are, so no token-to-token correspondence exists between GPT-2 and Pythia; passage-to-passage correspondence does. T also varies per passage, so per-token states do not form a rectangle, and both CKA and ridge need $[N, 768]$ matrices with row i the same passage in every model. Pooling moves the comparison to the level where alignment is real.

GPT-2 has no summary token. BERT and DINOv2 prepend a CLS token trained to represent the whole input; GPT-2 does not, because its attention is causal - a prepended token would see nothing, being first in line. Its only fully-contextual position is the last token, which over-weights the ending, so masked mean is the standard choice. Note the consequence: GPT-2 never produced a sentence-level vector during its own training, which is the literal sense in which it was 'never optimized for similarity'.

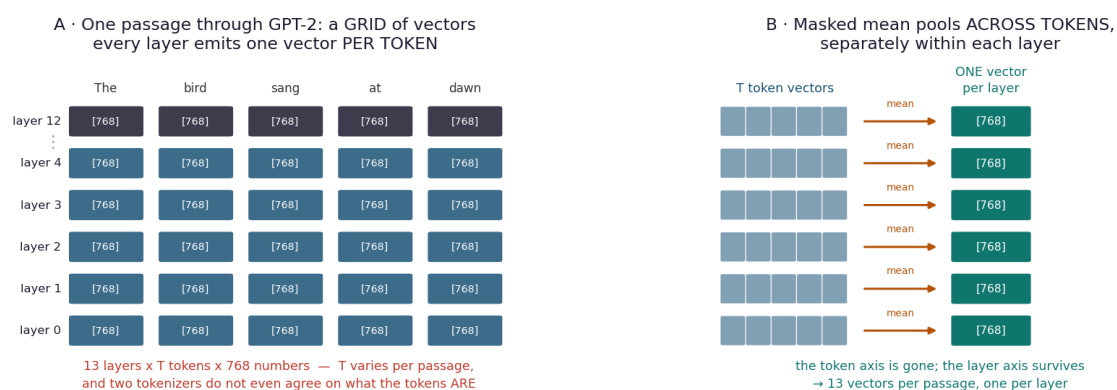


Figure A2: the token-by-layer grid, what masked mean collapses, and what each axis of the saved array means.

A8. The model roster: why each one, in which test, and what it proved

Nine pretrained models appear in this project. None was ever fine-tuned - every one is frozen, and every map between them is a closed-form ridge solve. Each was chosen to make one specific comparison clean.

model	role / where	why THIS model	what it let us prove
GPT-2 (124M)	Exp A source; E1.1/E1.2/E1.3 text target	causal LM, trained ONLY to predict the next token; never saw a similarity objective, and has no CLS token	that correspondence exists in raw LM representations (R2 retains 71%) and that its unreadability is geometric, not semantic - whitening recovers R@1 0.130 to 0.478
Pythia-160M	Exp A partner	same size class, DIFFERENT tokenizer, DIFFERENT positional scheme (rotary vs learned) and a different training corpus	stitching R2 0.737-0.798 vs 0.17 random: convergence survives genuinely independent training, not just seed variation
BERT-base	E1.3 ladder rung	masked LM, bidirectional - the objective BETWEEN causal LM and contrastive, so it tests the middle of the prediction	eff.rank 65.0 and R@1 0.362 landed exactly between GPT-2 and SBERT as pre-registered: rank-corr +1.00
SBERT (all-mpnet-base-v2)	E1.3 ladder rung	MLM plus contrastive - isolates what the contrastive stage adds over masked LM alone	the residual 0.110 spread after whitening: isotropy is necessary, NOT sufficient
bge-m3	E1/E1.3 reference text target	heavily contrastive over 1.2B text pairs - the purpose-built similarity space, i.e. the ceiling for a text target	the reference every whitened arm is scored against (102.6% / 101.4% / 111.8% / 126.2%)
DINOv2 small/base/large	E1 scale sweep, E1.2, G1/G2	SAME family and training recipe at 22M/87M/304M, and NO language supervision at any point - so capacity is the only variable and the cross-modal test is honest	alignment rises with capacity: 40.9 / 53.5 / 58.3% of ceiling, +15.3 points per decade, monotone in six measures
MobileCLIP-S1	Exp B / D phone tier	a real deployable on-device encoder, 512-d - the practical half of the two-tier system	the shipped 0.79 MB adapter: 95% of ceiling, verification AUC 0.999, membership AUC 0.889
SigLIP 2 (base-16-224)	Exp B / D server tier + ceilings	a strong jointly-trained image-text model, so its own image-caption retrieval IS the ceiling any adapter is measured against	the ceiling (R@1 0.630 same-modality, 0.871 cross-modal) and the shared index the phone tier writes into

The pattern behind the choices. Every pair differs in exactly the way the question needs and is matched everywhere else. GPT-2 vs Pythia isolates independent training. DINOv2 small vs base vs large isolates capacity. GPT-2 vs BERT vs SBERT vs bge-m3 isolates the training objective. MobileCLIP vs SigLIP isolates the deployment tier. Where a factor could not be held fixed - pooling in an early base run, caption protocol in the first ceiling - the affected run was excluded and re-measured rather than compared across.

Space geometry: what isotropy, anisotropy, rank collapse and whitening actually are

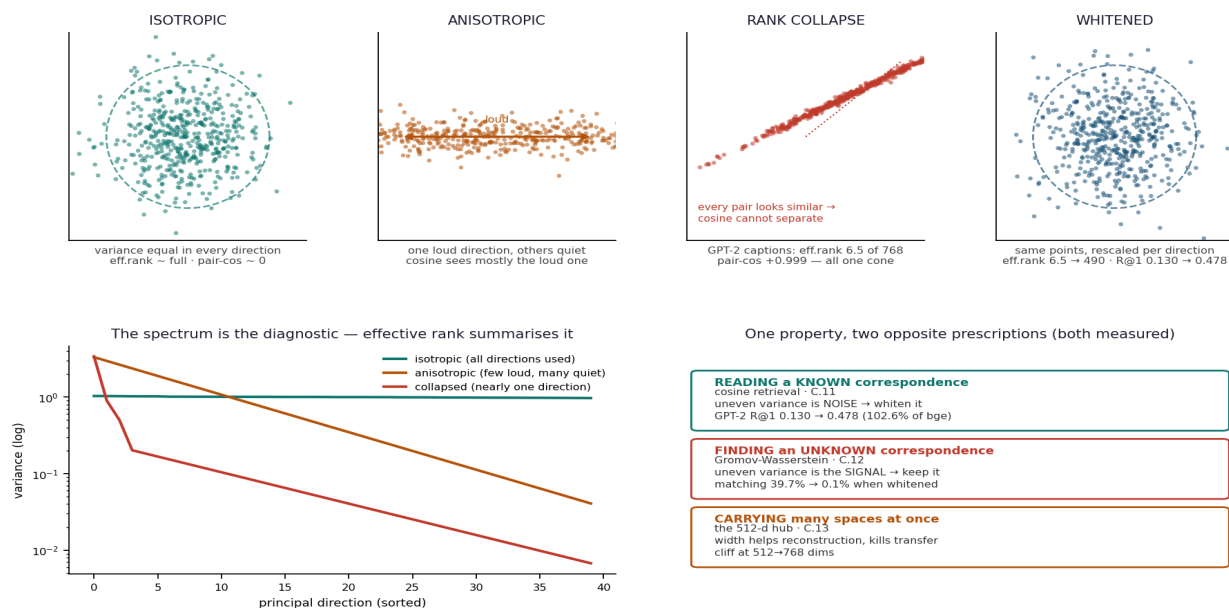


Figure A3: the four geometric states this project measures, the spectrum that diagnoses them, and the three settings where the same property demands opposite treatment.

A9. The GPT-2 collapse, in one place

Three findings turn on one phenomenon, so it is worth stating precisely. GPT-2's mean-pooled caption vectors have an effective rank of **6.5 out of 768** and a mean pairwise cosine of **+0.999** : a single direction carries 59.6 percent of all variance, and every caption vector sits in a cone a couple of degrees wide. Cosine similarity between any two of them therefore reads near 1.000 whatever they say.

Why it happens. GPT-2's only training objective is next-token prediction. It has no CLS token - its attention is causal, so a prepended summary token would see nothing - no pooling layer, and no similarity loss. It never produced a sentence-level vector during training at all; the sentence vector is our construction, a mean over token states optimised for a different task. Averaging then surfaces the large component every passage shares, and that component dominates.

Why it matters three times over. (1) It produces the project's cleanest case of metric degeneracy: the fitted map scores cosine 1.000 while retrieving at R@1 0.160, with held-out R-squared 0.414 - the content IS predicted, at 71 percent of the bge arm, and cosine simply cannot read it. (2) It is fully correctable by a label-free linear transform: whitening the target raises effective rank to 490.8 and R@1 to 0.478, which is 102.6 percent of an encoder contrastively trained on 1.2 billion text pairs (Section C.11). (3) It is the bottom rung of the pre-registered ladder in Section C.14, where raw retrieval across four training objectives ordered by effective rank with rank correlation +1.00.

The nuance to keep. Centering alone drives pair-cosine from +0.999 to -0.019 - the textbook symptom, apparently cured - while leaving effective rank at 6.5 and retrieval slightly WORSE (0.130 to 0.104). The variance was concentrated, not merely offset, so the remedy is rescaling rather than translation. Pair-cosine on its own is not a sufficient diagnostic; read it alongside a spectral measure.

The GPT-2 collapse: a space that encodes the content and hides it from cosine

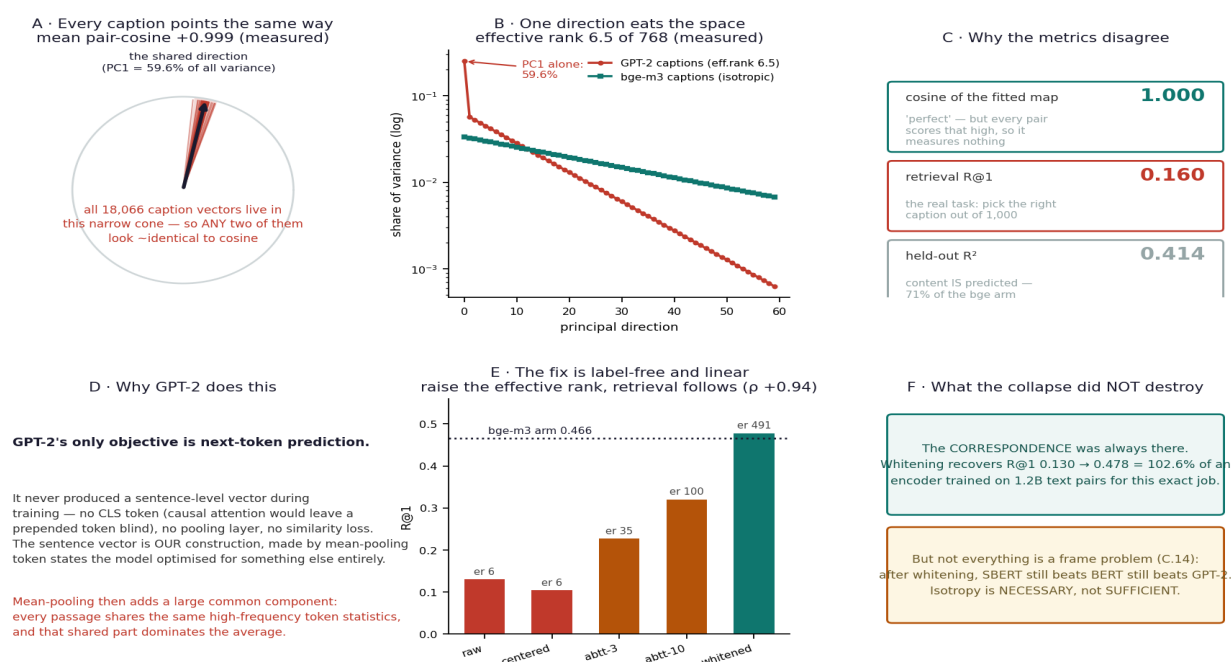


Figure A4: the collapse (the cone and the spectrum), the metric disagreement it causes, its origin in the training objective, the label-free fix, and what it did not destroy. Point clouds are schematic; every number shown is measured.

A10. Reading a collapse: margin, hubness, and what R@1 does not prove

Verification and retrieval ask different questions. Verification asks whether a true pair scores above a RANDOM pair - $P(\text{score_true} > \text{score_random})$, which is what AUC estimates. R@1 asks whether the true partner beats EVERY other candidate in the gallery. The second is strictly harder, and the gap between them is informative rather than contradictory. A worked case: the true pair scores 0.80, most impostors score 0.30 to 0.60, but two impostors score 0.83 and 0.82. Verification AUC stays high - the true pair still beats a typical random one - while R@1 fails for that query. This project measures exactly that pattern (Appendix D.2): verification AUC 0.999, per-query ranking 0.987, membership under one global cutoff 0.889.

Margin, not absolute cosine, is what a collapse destroys. It is possible for the true pair's similarity to RISE while retrieval gets worse, because the impostors rise faster. Before: true 0.85, random 0.20 - a margin of 0.65.

After a collapse: true 0.92, random 0.88 - a margin of 0.04. The first number improved and the representation became useless. Always report the separation, not the level: this is the same lesson as the caption-verification case where positives average cosine 0.136 against negatives at -0.024 and AUC is nonetheless 0.997.

Hubness is a SECOND pathology, distinct from anisotropy. Anisotropy means the vectors crowd into similar directions. Hubness means a few particular candidates become the nearest neighbour of disproportionately many queries - a property of the neighbourhood graph, not of the variance spectrum, and one that grows with dimension. The standard statistic is the skewness of N_k , the number of times each gallery item appears in some query's top-k. Measured here on 1,000 held-out images with $k = 10$, where every item would appear 10 times under a perfectly even distribution:

query space	N10 skewness	top 1% of items absorb	most-hubbed item	items never a neighbour
adapted (phone -> W)	1.61	4.6%	71 queries	0.1%
raw (no adapter)	4.28	10.9%	202 queries	18.2%
ceiling (SigLIP native)	1.41	3.9%	65 queries	0.0%
random-direction queries	0.91	2.9%	38 queries	0.1%

The raw cross-tier space is severely hubbed: one image is the nearest neighbour of 202 of 1,000 queries while 18 percent of the gallery is never anyone's neighbour. The adapter removes almost all of it - skewness 1.61 against the ceiling's 1.41 - so the map does not merely align the two spaces, it restores a healthy neighbourhood structure. This is worth stating because a hubbed space can pass verification while failing retrieval: a handful of hub vectors outrank the true partner for many queries at once.

Two causes of a collapse, and how to tell them apart. An arm can lose discriminability for reasons that are **INTRINSIC** to the encoder - GPT-2's caption space already has effective rank 6.5 of 768 before any mapping (Section A9) - or **INDUCED** by the pipeline, when a projection or hub too narrow to carry the distinguishing directions truncates them away. The diagnostic is to measure the geometry **BEFORE** the mapping and again after: if effective rank and N_k skewness are already pathological in the source, the cause is intrinsic; if they degrade only after projection, the pipeline caused it. This project has one measured instance of the intrinsic case - GPT-2's native collapse in Section C.11. The hub's own transfer cliff past 512 dimensions (Section C.13) is pipeline-induced in the sense that widening the hub destroyed transfer while rows per dimension stayed above threshold, but the **MECHANISM** is not established: both a noise-amplification account and a local-structure account were tested and falsified (C.13.11). Do not cite it as a worked example of induced truncation.

What a collapsed R@1 does NOT prove. It does not establish that shared structure is absent. It is equally consistent with structure being present but no longer **RESOLVED** at retrieval precision by the current mapping or geometry. The tell is a surviving verification AUC: coarse correspondence retained, pair-specific discriminability lost. Section C.11 is the decisive case - retrieval had collapsed to R@1 0.130 while held-out R-squared stayed at 0.414, and a label-free linear transform then recovered R@1 to 0.478. Had the collapse been read as absence of structure, the finding would have been missed.

A6. Running the notebooks: runtime and storage

The notebooks run unchanged on Colab or on a local machine. Two independent choices, and they are often confused:

Choice	Options	What it controls
Runtime (where code executes)	Colab hosted VM (T4 / L4 / CPU) local kernel	Chosen in the Colab menu (Runtime -> Change runtime type), or simply by opening the file in Jupyter. No code selects it. To use your own GPU inside the Colab interface: install <code>jupyter_http_over_ws</code> , launch jupyter with <code>--NotebookApp.allow_origin='https://colab.research.google.com'</code> , and paste the printed <code>http://localhost:8888/?token=...</code> into Connect -> Connect to a local runtime
Storage (where data lives)	STORAGE = 'drive' (default) 'local' 'env'	'drive' mounts MyDrive/convergence_experiment - survives session timeouts, so checkpoints resume. 'local' uses LOCAL_DIR on whatever machine is running; on a hosted Colab VM that disk is ERASED at session end, so downloads restart from zero. 'env' honours a DATA_DIR already set. On a local kernel, 'drive' falls back to LOCAL_DIR automatically and says so - Drive mounting is a Colab-VM feature and cannot work locally

Which to use. The dominant cost in the E-series is downloading COCO images, not GPU time: a hosted VM has datacenter bandwidth, a local machine has your uplink, so hosted can finish the fetch faster even with a weaker GPU. Against that, a local kernel has no quota and no session timeout - which matters most for the largest sweep point (dinov2-large at cls+patch, 23,755 images). Because checkpoints are plain .npz validated against the current configuration, the two can be mixed: fetch on a hosted runtime writing to Drive, then continue locally with STORAGE='env' pointed at the same folder synced by Google Drive for Desktop.

A free T4 is sufficient for dinov2-small and -base; CPU works but turns minutes into hours, and the first cell prints a warning when no GPU is found. The cell also prints which environment, device and DATA_DIR it resolved, which is the quickest way to confirm a local runtime actually took effect - it reports 'local machine' rather than 'Colab'.

A5. What is and is not trained

Series	Trained?	Detail
A, B, E	No	Models are downloaded frozen and run under no_grad; the only fitted object is a closed-form ridge matrix outside both models. Weights are bit-identical to the downloads afterwards - delete W and nothing remains
F2	Yes	Trains a separate MLP head <i>beside</i> the frozen encoders; the encoders themselves are untouched
F3	Built, deliberately unrun	The only notebook that would modify a pretrained model's own weights (MobileCLIP's last stage). Gated on F2's verdict and on whether moving the ceiling is worth forfeiting the seconds-long refit

One GPU pass, then a ruler. The only GPU-bound step in the whole project is the one-time encoding pass that builds the representation caches - the eight encoders run forward over 9,533 items once (T4/L4 on Colab; CPU works but turns minutes into hours). Everything downstream is closed-form linear algebra that runs on CPU: the ridge maps, the hub, and all transfer. The expensive hardware builds the raw material once; the science runs without it - the same economy as the 0.79 MB adapter that ships and runs with no GPU at all.

A11 · The model roster: what each encoder is and why it is here

Every model is pretrained and frozen. What differs is modality, training objective, lineage and the geometry of the space it produces - and those four things are exactly the variables the experiments vary.

model	modality	objective	lineage	space (eff. rank / pair-cos)	role and why it was chosen
GPT-2	text	causal LM	OpenAI, WebText	768-d; 6.6 / +0.999 - collapsed	Exp A partner (vs Pythia); text target in E1.1; the collapsed case that drove C.11 and the read-only finding. Chosen because it was never trained for similarity - the hardest text case
Pythia-160M	text	causal LM	EleutherAI, The Pile	768-d, healthy	Exp A partner. Different lab, tokenizer, corpus, positional scheme from GPT-2 - so any agreement is non-trivial
MobileCLIP-S 1	image	contrastive image-text	Apple	512-d; eff. rank 140.9, mean-dir ratio 0.43	The PHONE tier of Exp B; adapter source. Small enough for on-device Core ML
SigLIP 2	image (+text tower)	contrastive image-text (sigmoid)	Google	768-d; eff. rank 181.7	The SERVER tier and Exp B target; teacher for F1/F2; G4 cross-lineage encoder (no DINOv2 ancestry)
DINOv2 S/B/L	image	self-supervised, no text ever	Meta, distilled from one ViT-g	768/1536/2048-d; healthy	The E-series image side and the hub's image encoders. Never saw language, so cross-modal agreement cannot be trained-in. Three scales share a teacher - the CLEAN capacity control, and the lineage caveat
bge-m3	text	heavy contrastive (text-text)	BAAI	1024-d; 97.4 / +0.572	The readout target for every cross-modal number - retained for COMPARABILITY, not merit (SBERT is marginally better)
BERT	text	masked LM	Google	768-d; 65.0 / +0.806	Middle rung of the E1.3 ladder; hub member. Never trained for similarity - tests whether MLM beats causal LM on geometry
SBERT	text	MLM + contrastive	UKP / sentence-transformers	768-d; 93.4 / +0.139	Upper rung of the ladder; hub member; the best-behaved target (101.0% writable). Shares an OBJECTIVE with bge but no lineage - the pair that agrees most

Table A11: the roster. The last column is the reason each model earned its place; the geometry column is what predicts how it behaves.

A12 · The measurements: what each metric asks, and how to read its answer

The project's central lesson is that a metric is only interpretable relative to the geometry it is computed in. This table states, for every measurement used, what question it answers, what breaks it, and what a high or low value means here.

metric	what it asks	invariant to	BREAKS when	read as
cosine	do two vectors point the same way?	scale (L2-normalised)	the space is collapsed - reads +0.999 for everything	the index's own similarity; meaningless without knowing the space's pair-cosine baseline
R2 (held-out)	do predicted coordinates match the target's, number by number?	nothing - it is coordinate-wise	target has tiny variance (SST $\rightarrow$ 0) $\rightarrow$ hugely negative on collapsed targets	reconstruction fidelity WITH magnitude. Untrustworthy for collapsed targets - use R@1
R@1 / R@5 / R@10	does the predicted vector retrieve the RIGHT item among N?	scale; monotone transforms	gallery size changes - the number means nothing without N (0.967 @10, 0.557 @1000)	the deployed question. Always vs chance 1/N and vs a ceiling measured the same way
% of ceiling	how much of a jointly-trained system's retrieval survives?	-	the ceiling is measured under a different protocol	the honest cross-experiment yardstick; ceiling always re-measured per run
CKA	are two spaces' similarity STRUCTURES alike?	rotation, scale, permutation	very different tokenisers depress absolute values	compare the RATIO to a random control (6.3x), not the absolute (0.424)
Procrustes R2	is the map a pure ROTATION?	-	spaces differ in stretch, not just orientation	low vs ridge = anisotropy mismatch (0.038 vs 0.592). Not a failure - a diagnosis
effective rank	how many directions carry variance? (spectral entropy)	rotation	-	<10% of width = collapsed. Predicts writability and cosine validity better than anything else
mean pair-cosine	how similar are RANDOM pairs of items?	-	-	isotropic $\sim$ 0; +0.999 = collapsed. The single fastest diagnostic
Spearman rho of pairwise distances	do two spaces AGREE on how far apart items are? (no map)	rotation, scale, dimension, monotone warps	-	Shape agreement before any fit. Rank-based, so collapse does NOT depress it (tested)
kNN overlap / chance	are my nearest neighbours your nearest neighbours?	everything except neighbourhood	raw fraction misleads - rises with k because chance does	read the RATIO to k/N. Falling with k = local structure; flat = global
N10 hubness skew	do a few items dominate everyone's neighbourhoods?	-	-	<2 healthy; >3 hubbed. Adapter took raw 4.28 $\rightarrow$ 1.61
verification AUC	can a threshold tell same-item from a TYPICAL impostor?	-	-	separation, not magnitude - small cosines with a clean split still give $\sim$ 1
membership AUC	can ONE global cutoff tell indexed from not-indexed?	-	negatives are the BEST of a gallery, so they sit high	always below verification AUC; the gap is the extreme-value effect
rows per input dim	is the fit powered?	-	<5 $\rightarrow$ starved; negative R2 on a control is the signature	check BEFORE believing any negative result

Table A12: read the 'BREAKS when' column before trusting any number in the project - it lists every metric artifact that was actually hit.

A13 · Why GPT-2 behaves differently in the G-series - and why that is a result

GPT-2 appears in three G-series roles and behaves differently in each. All three follow from one property of its caption space: effective rank 6.6 of 768 and mean pair-cosine +0.999. Every item sits almost on top of every other.

role	what happens	number	why
as a SOURCE (read out of)	works fine	95.1% retention	reading out needs only the CONTENT, and the hub keeps it. Rank-based structure survives collapse
as a TARGET (written into)	fails	24.8% retention	ranking INSIDE a space whose items are +0.999 apart needs precision in the last decimal; one extra fitting stage costs more than the gaps are wide
meeting IN the hub (G3)	works, and is RESCUED	raw 0.104 -> hub 0.458 (4.4x)	the whitening inside the hub basis is C.11's isotropy fix, applied for free. Encoders never rebuild GPT-2's geometry, they meet in shared coordinates
shape agreement (no map)	lowest of all seven	rho 0.17-0.25	on synthetic data collapse did NOT depress rank correlation (tested); on the real encoder it is lower - so GPT-2 encodes LESS of the shared structure, not just less readably

Table A13: the same space, four behaviours, one cause.

Three explanations were tested and two were wrong. 'The hub discards GPT-2's directions' - false, the hub reconstructs it at +1.000. 'Low reach-into-hub predicts poor writability' - false, BERT reaches the hub less well than GPT-2 and is three times more writable. What survived: target self-similarity (+0.999) makes within-space ranking need precision an extra stage cannot preserve - reproduced in a controlled experiment, 152% -> 31% retention when a healthy target was collapsed to the GPT-2 signature.

Why it does not bind deployment. A shared index means encoders meet **IN** the hub (G3's question, 42/42 shared) - you never need to regenerate GPT-2's native vectors. And GPT-2 reaches the bge readout at 93.9%, so it can drive the caption head. The limitation is precise and it is a property of the target, not the method.

A14 · Local versus global similarity, and which Platonic claim survives

The question. When two encoders 'agree', do they agree about the whole space or only about neighbourhoods? These are different claims with different consequences, and the project measured which one holds.

The measurement. For each item take its k nearest neighbours in each of two spaces; count the overlap; divide by chance (k/N). Repeat for k from 1 to 500. If the ratio holds, structure is shared at every scale; if it falls, agreement is local.

scale	k	typical ratio to chance	reading
local	1	70-970x	strong: encoders agree on nearest neighbours
mid	50	7-24x	fading
global	500	1.3-2.3x	near chance: little global agreement

Table A14a: raw ratio to chance. This is NOT the locality evidence - the ratio is bounded by N/k (C.13.9). Chance-corrected, agreement PEAKS at $k \sim 20-50$; CKNNA carries the verdict.

At what scale do the encoders agree?

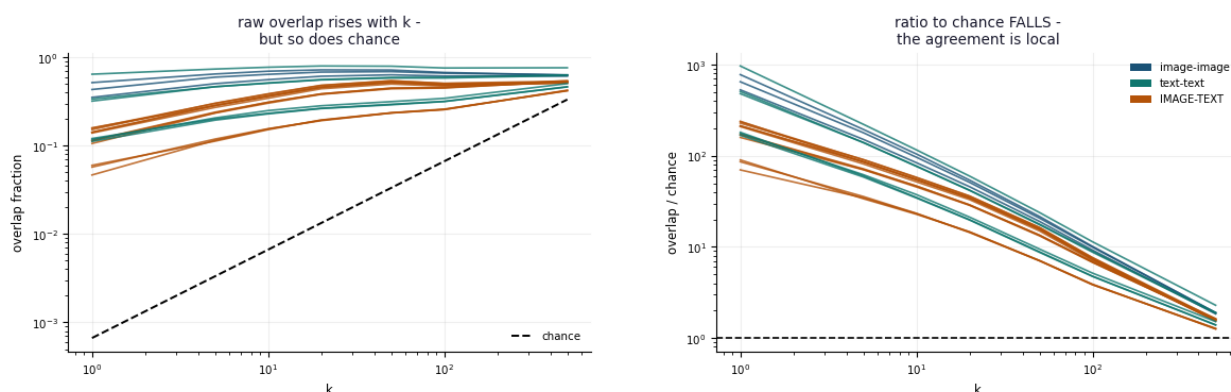


Figure A14a: raw overlap rises with k only because chance does (left); the ratio to chance falls for every pair (right).

The proof it is local. The right panel is the whole argument: every line dives toward the chance floor. That is the signature of locally shared, globally divergent structure - two map projections preserve adjacency and distort distance.

Local topology: which neighbour relations survive across encoders

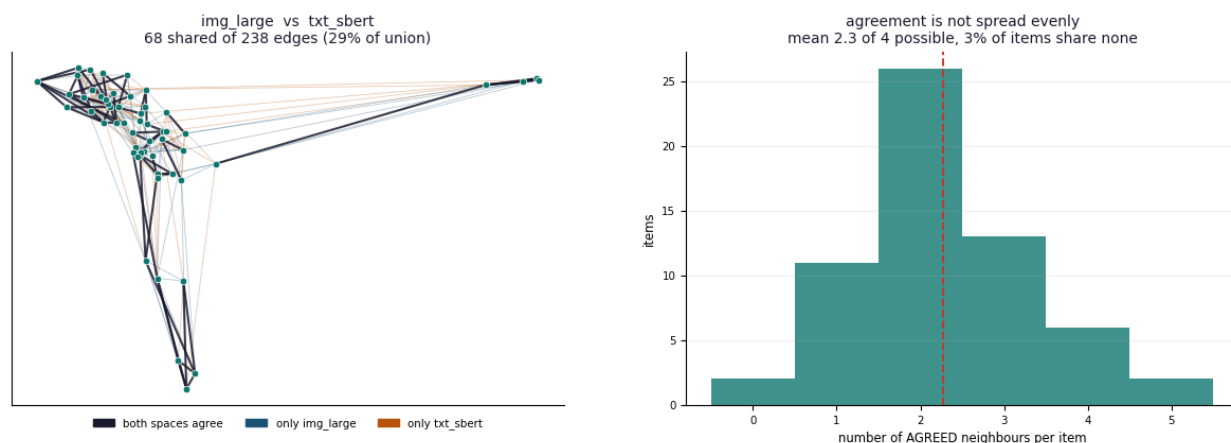


Figure A14b: dark edges are neighbour relations BOTH an image encoder and a text encoder agree on, with no map fitted - 68 of 238 (29% of the union). Positions are arbitrary; edges are the measurement.

Why this is still Platonic - the precise version. The naive Platonic claim, 'models converge to the SAME space', is REFUTED: raw cosine is zero and global agreement is near chance. The precise claim, 'models are different CHARTS of one shared relational manifold', is SUPPORTED: coordinates differ entirely, local relations survive the change of chart. Every result in the project is consistent with the chart reading and it resolves what would otherwise contradict - linear maps work (some structure is global) but miss part of the ceiling (the richest agreement is local, and no linear map carries local structure globally); Gromov-Wasserstein recovers pairings because it reads exactly the local distances that are most shared; capacity scaling is the Platonic prediction in testable form. The claim is narrowed to what the measurements support, and no further.

A15 · The drawings that show shape agreement

Two figures carry the visual argument that structure is shared before any map exists.

Geometry similarity: the same shape in two unrelated coordinate frames



Figure A15a: pairwise distances agree (A, $\rho = +0.681$) and neighbourhoods agree (B, 67% vs 2.5% chance) with NO fitted map; the clouds nonetheless sit in unrelated frames (D) until one linear map lands each phone vector on its twin (E).

Shape agreement across modalities — seven encoders, four training objectives, no fitted maps

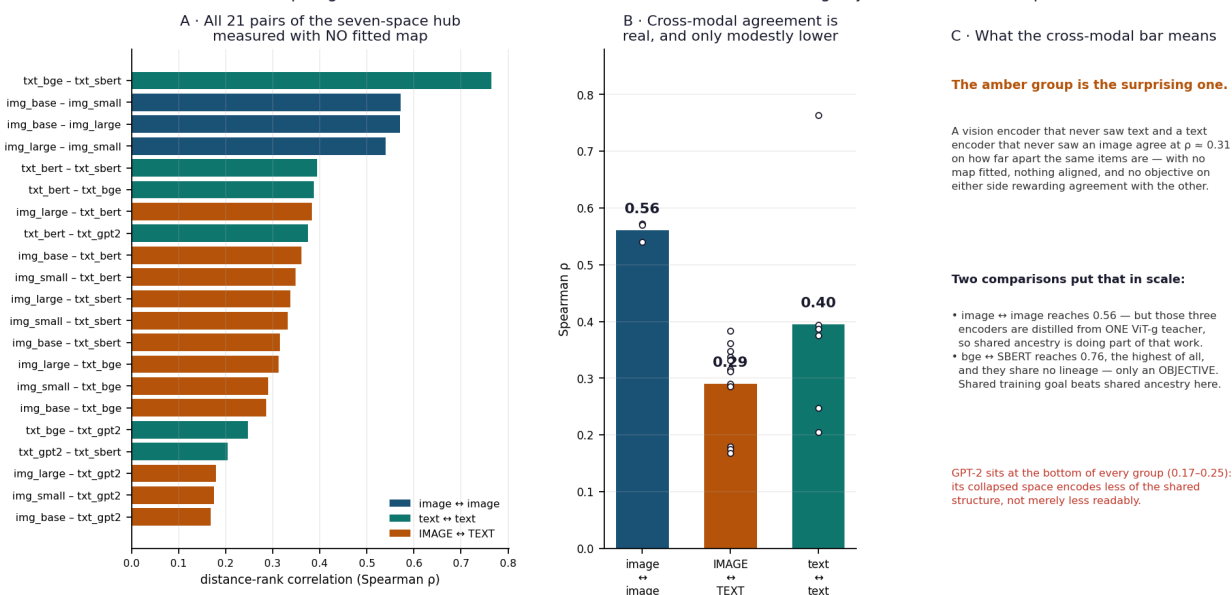


Figure A15b: all 21 pairs of the seven-space hub, no map fitted. Cross-modal pairs (amber) agree at $\rho \sim 0.29$; the highest pair of all is bge-SBERT at $+0.76$, sharing an OBJECTIVE and no lineage.

A16 · One worked example, carried through every metric: a cat on a chair

The metrics in A12 are abstract. This section runs a single semantic item through all of them, so each measurement has a concrete referent. The item is a photograph of a cat sitting on a chair, paired with the caption "A cat is sitting on a chair." Three encoders give it three unrelated coordinate vectors; the question is never whether the numbers match, but which RELATIONS survive.

```
semantic item i = [image: a cat on a chair] + [text: "A cat is sitting on a chair."]
```

```
DINOv2 image_i -> x_img in R^768 (never saw the caption)
bge-m3 text_i -> x_bge in R^1024 (never saw the image)
GPT-2 text_i -> x_gpt2 in R^768 (collapsed: eff. rank 6.6)
HUB both -> h_i in R^512 (shared coordinates)
```

The neighbourhood is the invariant. Around the cat-chair item, a well-behaved space should place other cats and other furniture near, and unrelated concepts far. That ordering is what survives a change of encoder, even though the coordinates do not.

```
NEAR (should stay near in every space):
  cat on a sofa · kitten on a chair · pet on furniture · cat indoors
FAR (should stay far in every space):
  an airplane · a spreadsheet · a volcano
```

metric	what it asks OF THE CAT-CHAIR ITEM	result in this project
kNN overlap	are cat-sofa and kitten-chair among its neighbours in BOTH the image and the text space?	yes, strongly, at small k - 100s x chance at k=1; the agreement fades as the neighbourhood widens
distance rank	does it rank closer to cat-sofa than to airplane in both spaces?	yes - rank correlation up to +0.76; collapse does not break this (rank-based)
ridge map	can the image vector of the cat be mapped to the direction of its caption?	yes - R@1 up to 58% of ceiling; this is the linear, exportable part
Procrustes	after PCA, does a pure rotation align the image cat-chair with the text cat-chair?	no - 0.038 vs ridge 0.592; the spaces differ in STRETCH, not just orientation
R@1 vs AUC	is the true caption the single top match, vs merely above a random airplane?	AUC stays high (above a typical negative); R@1 can fail when a few near-duplicate captions outrank it - the GPT-2 case
hub transfer	can a head trained on DINOv2-small caption THIS cat when fed a different encoder's hub vector?	yes - 93-96% of native, including a cross-lineage encoder at 94.2%

Table A16: one item, every metric. Read down the middle column to see what each measurement actually asks; read the right column for what the project found.

Why the cat keeps its neighbours but not its coordinates. DINOv2 and bge-m3 never trained together and share no weights, so `x_img` and `x_bge` live in unrelated frames - raw cosine between them is meaningless. What both encoders learned, independently, is that a cat on a chair belongs among cats and among furniture and far from airplanes. That neighbourhood is a property of the world, not of either coordinate system, which is why it survives the crossing. The coordinates are each encoder's private chart; the neighbourhood is the shared territory.